ros2_shm_fanout

A Shared-Memory Transport for High-Fan-Out Perception

Systems / Robotics Middleware Project · Raj Priyadarshi

Abstract

When streaming high-bandwidth RGB-D camera frames from a robotics/simulation pipeline (NVIDIA Isaac Sim) to multiple vision models, ROS 2's default message-passing transport becomes a bottleneck: each subscriber receives its own copy of every multi-megabyte frame, so producer cost and bandwidth scale linearly with the number of consumers. The production zero-copy paths further require every consumer to be a configured DDS participant — impractical when vision models run in separate conda environments.

ros2_shm_fanout is a purpose-built, single-writer / many-reader shared-memory transport that writes each frame once into a named /dev/shm segment; every consumer mmaps that same memory and reads the pixels in place — zero copies on the read side, and no requirement that a consumer be a ROS 2 node at all. It was built to feed models such as YOLO, Florence, SAM, RT-DETR, GG-CNN, and GraspNet running across independent Python environments.

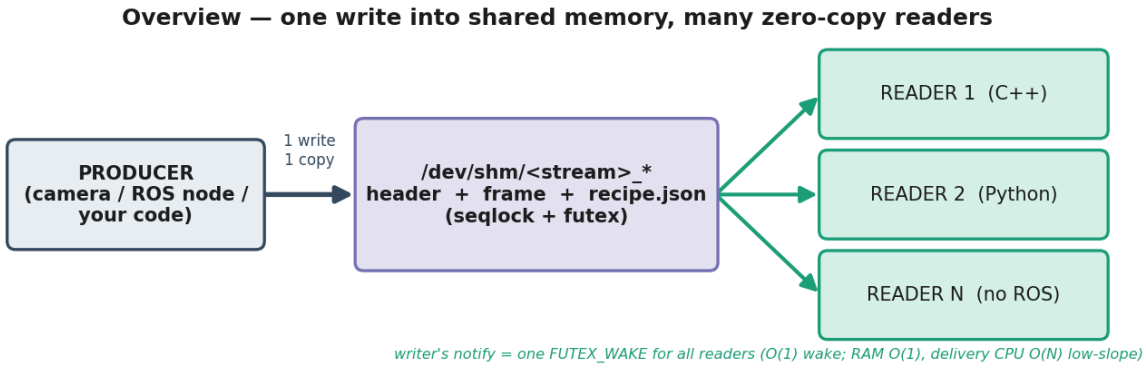
Technical Highlights

- **Zero-copy on read:** one physical copy of each frame is written to /dev/shm and mapped in place by every consumer — producer cost is flat in the number of readers (write once, map many).
- **Framework-agnostic consumers:** any process that can mmap a file can read the stream — no ROS link, no DDS participation, no middleware daemon — so vision models in separate conda environments attach freely.
- **Purpose-built RGB-D payload:** RGB (bgr8) plus float32 metric depth and a JET-colormapped depth view, with camera intrinsics carried in JSON sidecars for downstream deprojection and grasping.
- **Latest-frame-wins semantics:** a slow consumer can never apply backpressure to the live camera; readers always get the freshest frame.
- **Torn-frame guard:** a data-size-vs-buffer check protects readers against partially written frames.
- **Benchmarked against the ecosystem:** compared with Fast-DDS SHM, CycloneDDS+iceoryx, and ros2_shm_msgs using the Apex.AI performance_test methodology.

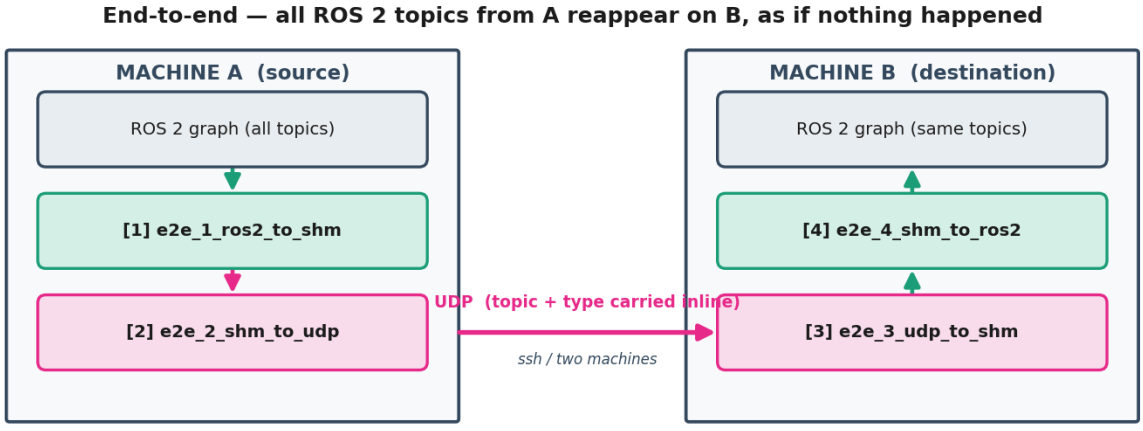
Technologies Used

- **Languages:** C++, Python
- **Middleware:** ROS 2 (rclcpp / rclpy)
- **IPC:** POSIX shared memory (mmap, /dev/shm), single-writer / many-reader design
- **Perception:** OpenCV, and downstream models (YOLO, Florence, SAM, RT-DETR, GG-CNN, GraspNet)
- **Benchmarking:** Apex.AI performance_test vs Fast-DDS SHM, CycloneDDS+iceoryx, ros2_shm_msgs
- **Environment:** NVIDIA Isaac Sim RGB-D perception pipeline across multiple conda environments

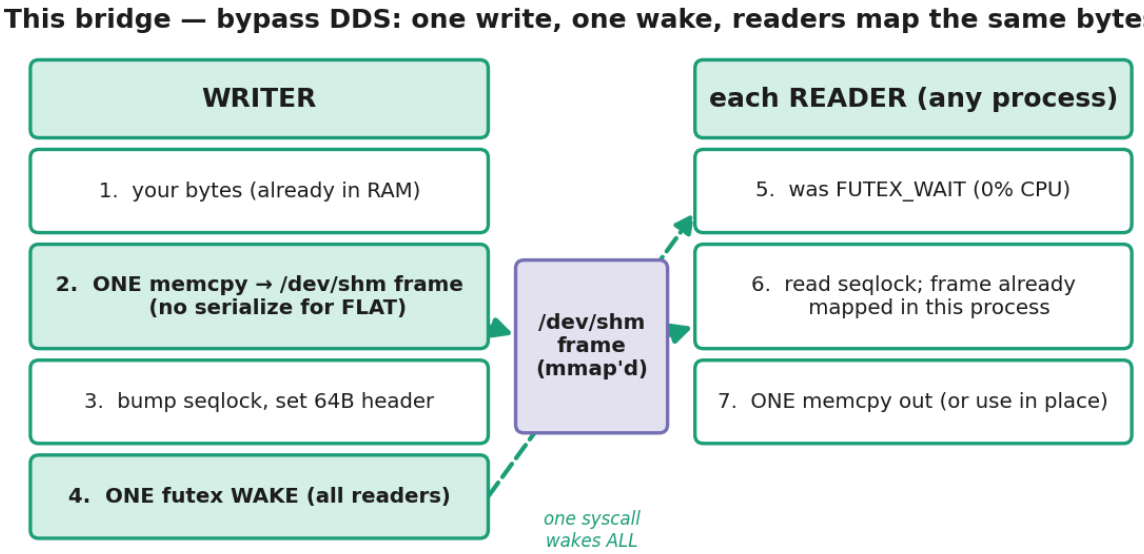
Figures



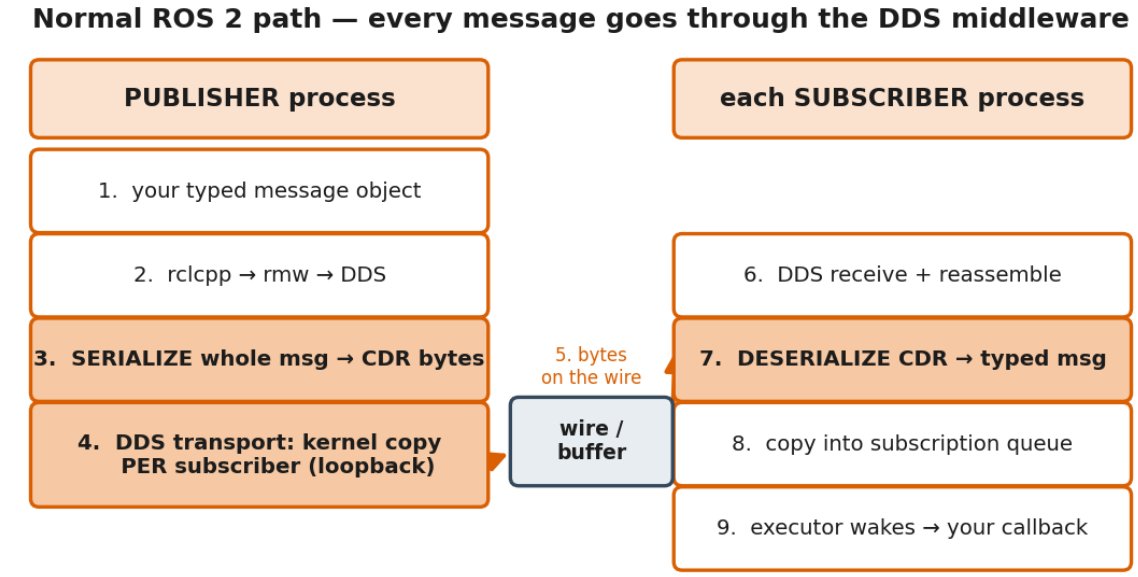
System overview of the shared-memory fan-out transport.



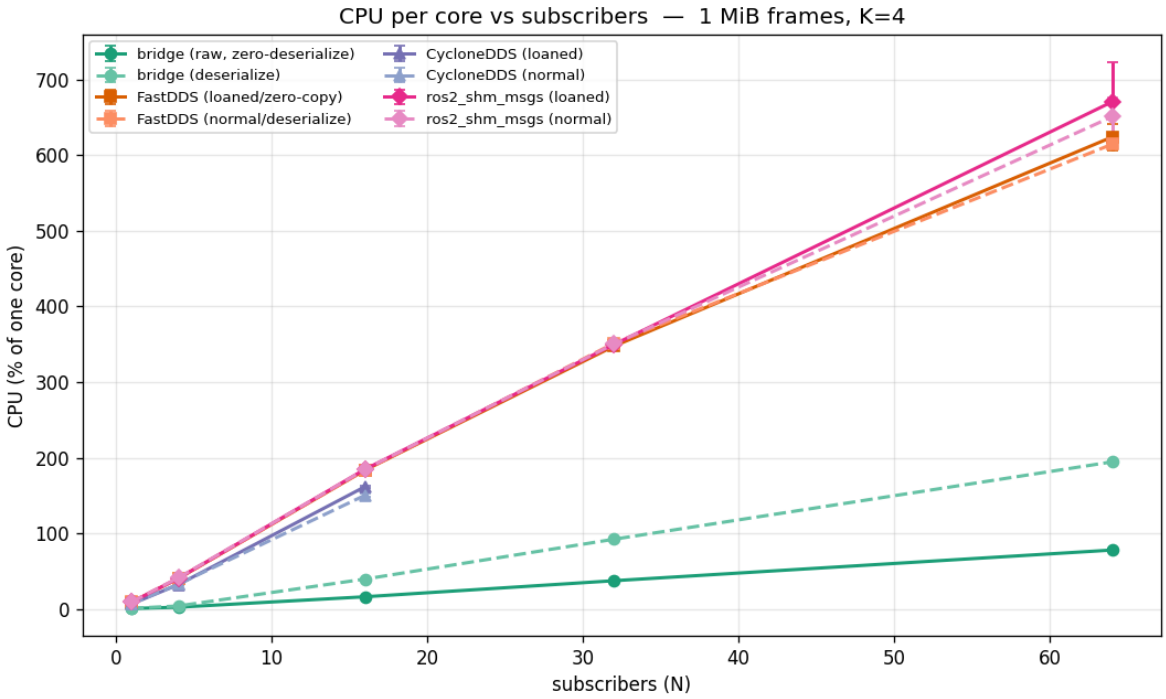
End-to-end perception pipeline the transport plugs into.



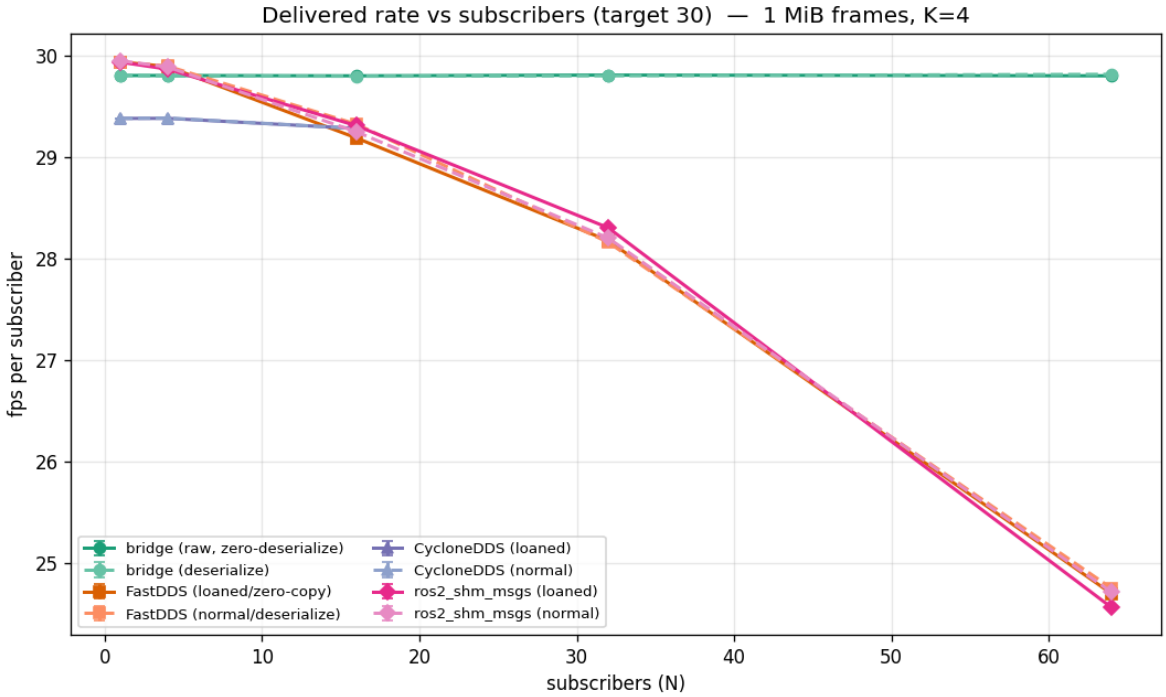
Shared-memory data path: one copy in /dev/shm, mapped in place by every reader.



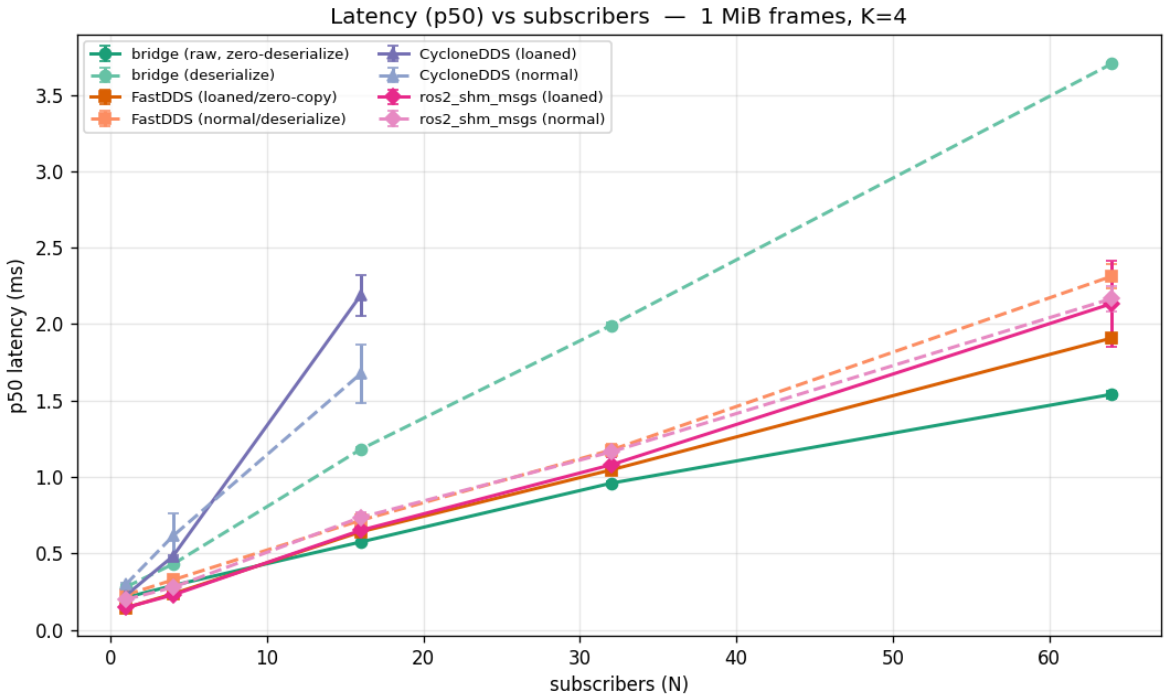
Default DDS data path for comparison: per-consumer serialization and copies.



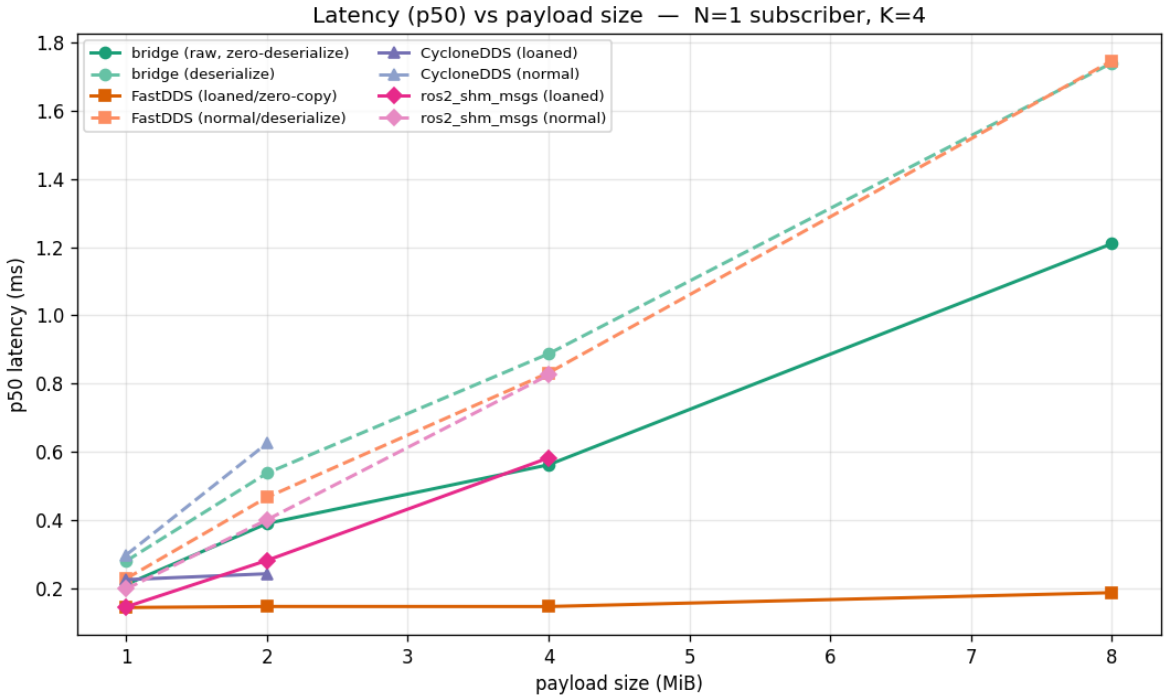
CPU usage vs subscriber count (1 MP frames) — near-flat for shared memory.



Sustained frame rate vs subscriber count — steady throughput out to 64 consumers.



Latency vs subscriber count.



Latency vs message size.

Outcome

ros2_shm_fanout turns a per-subscriber copy explosion into a single shared copy: the producer's cost stays flat as consumers are added, and any process — ROS or not — can read the freshest frame in place. It is a clean, practical transport for feeding many perception models from one high-bandwidth camera stream.